CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 4

Outline

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

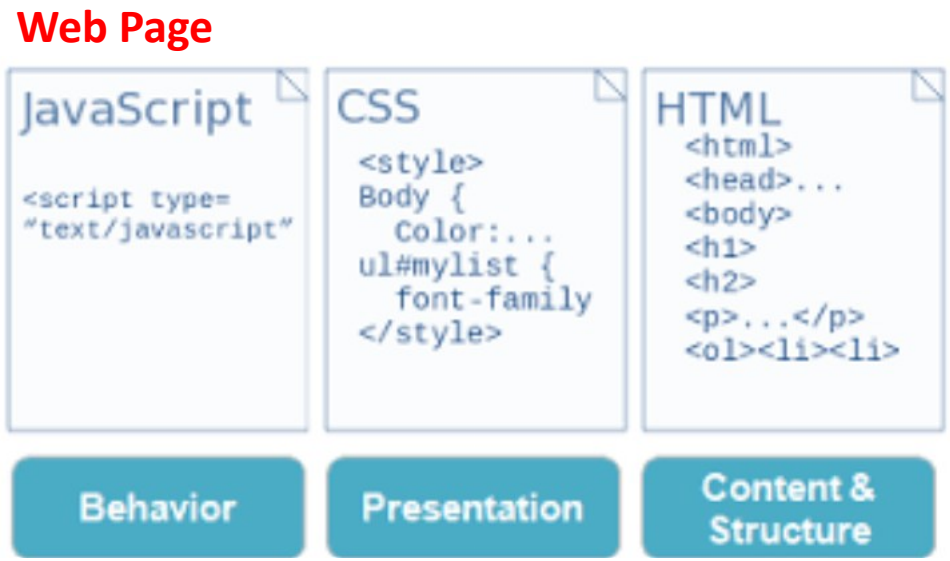
What is Java Script and Scripting Language

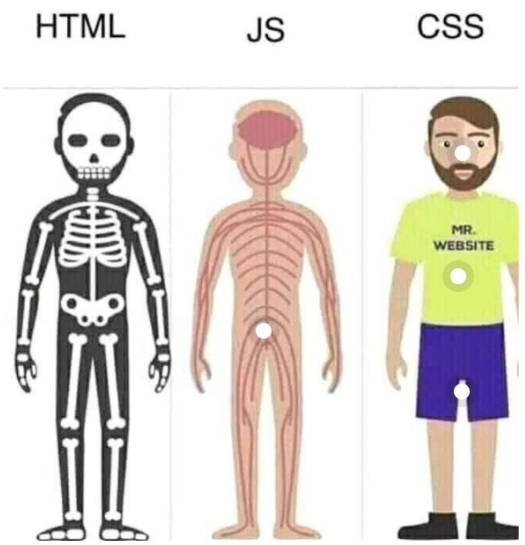
JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

What is JavaScript?

- JavaScript is a **scripting language** that allows you to implement complex features on web pages
- In JavaScript, the programs are called scripts





What is a Scripting Language

- A **scripting** or **script language** is a programming language for a special run-time environment that automates the execution of tasks
- The primary difference is in their execution:
  - Programming languages use a compiler to convert the high-level programming languages into machine language. Compiler compiles a code in a complete chunk
  - Scripting languages use an interpreter. An interpreter compiles a code line by line.

Java vs. JavaScript

- JavaScript is completely different from Java
- Initially, when JavaScript was created, it was called “LiveScript”. Then, as Java was extremely popular in that period, then it was decided to call it JavaScript to position it relative to Java.
- But, over the years, JavaScript has transformed into a completely independent language, with its specification, known as ECMAScript, having no relation to Java.
- Modern JavaScript can both work in the browser and on the server. Basically, it can run on any device that has a specific program known as the JavaScript engine (ex: Google’s v8 engine).

What can JavaScript do?

- With a programmable object model, JavaScript gets all the power it needs to create dynamic HTML.
- JavaScript can
  - change all the HTML elements and attributes in the page
  - change all the CSS styles in the page
  - react to all the events in the page
  - calculate, manipulate and validate data

What is Vanilla JavaScript?

- Vanilla JavaScript is the JavaScript language “as is” without using any tools to make the coding process easier or more efficient.

Ecma International

- An industry association dedicated to the standardization of information and communication systems

https://www.ecma-international.org/

ECMA - *European Computer Manufacturers Association*

Standards

- A standard is a document that provides rules, guidelines or characteristics for activities or their results, for common and repeated use.

  https://www.ecma-international.org/publications-and-standards/standards/

- Ex:

Number	Title
ECMA-262	ECMAScript® 2022 language specification
ECMA-419	ECMAScript® embedded systems API specification
ECMA-334	C# language specification
ECMA-404	The JSON data interchange syntax

ECMA-262

ECMA-262

ECMAScript® 2025 language specification

16th edition, June 2025

- This standard defines the ECMAScript 2024 general-purpose programming language.
  - While ECMA-262 is the name of the standard it represents the scripting language specification ECMAScript

https://www.ecma-international.org/publications-and-standards/standards/ecma-262/

ECMAScript Versions

- 1st Edition, 1997
- 2nd Edition,1998
- 3rd Edition 1999
- 4th Edition (unreleased)
- 5th Edition, 2009
- 6th Edition ,2015
- 7th Edition, 2016
- 8th Edition, 2017
- 9th Edition, 2018
- 10th Edition, 2019
- 11th Edition, 2020
- 12th Edition, 2021
- 13th Edition, 2022
- 14th Edition, 2023
- 15th edition, 2024
- 16th edition, 2025
- ES.Next

ES6 edition started a REVOLUTION and a new generation of JavaScript emerged, an advancement of great interest to any respectable frontend developer

ES.Next is a dynamic name that refers to whatever the next version is at the time of writing.

Major implementations

- JavaScript
- ActionScript
- Jscript
- SpiderMonkey
- V8
- QtScript
- InScript
- Google Apps Script

Overview of ECMAScript6…

https://www.javascripttutorial.net/es6/

- ECMAScript features
  - **Variables:** var, let, const
  - **OOP:** classes, super, get/set
  - **Functions:** generators, iterators, arrow functions, for-of
  - **Data Structures:** set/weakset, map/weakmap
  - **Async operations:** built-in promises
  - **Modules:** imports, exports, compatibility
  - **Objects:** computed properties, shorthand properties, Object.is(), Object.assign(), proxies
  - **Others:** templates, Math and Number extensions

Adding JavaScript to a web page

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

Adding JavaScript to a web page

- 3 methods:
  - JavaScript codes can be in a separate file
  - JavaScript codes can be embedded in HTML documents
  - JavaScript object attributes can be placed in HTML element tags

Method 1: Using External JS File Example

- Included in a .js file

HTML

```html
<html>
<head>
    <title>Week3 Samples</title>
    <script type="text/javascript" src="functions.js"></script>
</head>
...
```

Not required

functions.js

```js
function addToLocal() {
    ...
}
function removeFromLocal() {
    ...
}
```

Method 2: Embedded JS

Not required

```
<head>
   ...
    <script type="text/javascript">
        function myFunction(p1, p2) {
            return p1 * p2;
        }
    </script>
</head>
<body>
...
</body>
```

Method3: Using JavaScript in HTML element tags

```
<button onclick="alert('Hello')">Say Hello</button>

<button onclick="getElementById('demo').innerHTML = new Date()">

      What time is it?

</button>
```

Practice with **js-sample1.html**

JavaScript in <head> or <body>

- Scripts can be placed in the <body>, or in the <head> section of an HTML page, or in both.
  - But always reference CSS files within the <head> section
- <head> tags, signaling the browser to run the script before loading in the rest of the page.
- Placing scripts at the bottom of the <body> element improves the display speed, because script loading&interpretation slows down the display
  - Let the browser load the page first.
  - While a script is downloading, the browser will not start any other downloads. In addition, all parsing and rendering activity might be blocked.

HTML DOM

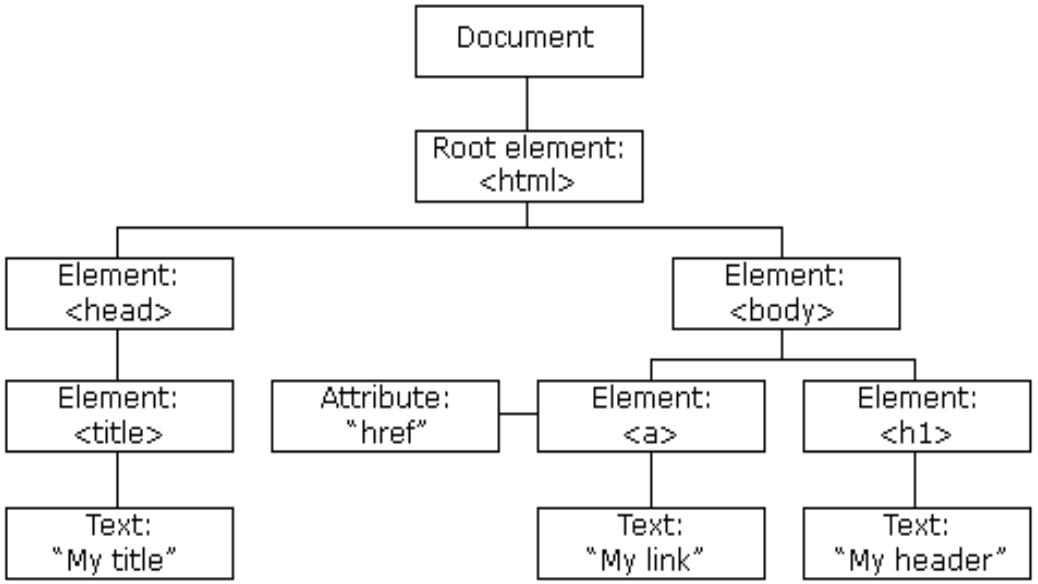
JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

The HTML DOM

- **DOM:** Document Object Model
  - Programming interface for HTML
  - Defines a standard for :
    - The HTML elements as objects
    - The properties of all HTML elements
    - The methods to access all HTML elements
    - The events for all HTML elements
- When a web page is loaded, the browser creates a Document Object Model of the page.

DOM Tree




```
<html>
  <head>
    <title>My Title</title>
  </head>

  <body>
    <h1>My header</h1>
    <a href="https://www.mylink.com.tr/">
       My link
    </a>
  </body>

</html>
```

DOM Tree

- According to the Document Object Model (DOM), every HTML tag is an object.
- Nested tags are “children” of the enclosing one.
- The text inside a tag is an object as well.

Nodes

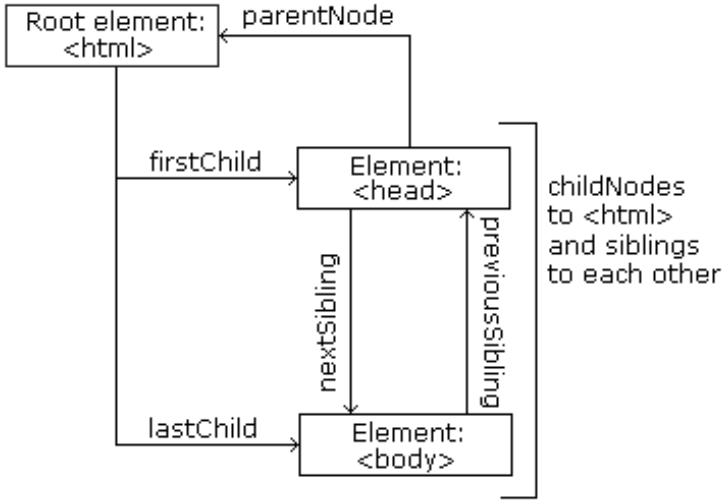
- All items in the DOM are defined as nodes.
- There are many types of nodes, but there are three main ones that we work with most often:
  - Element nodes
  - Attribute nodes
  - Text nodes
  - Comment nodes

Node Relationships

- The nodes in the node tree have a hierarchical relationship to each other.
  - The top node is called the root node
  - Every node has exactly one parent, except the root (which has no parent)
  - A node can have several children
  - Siblings are nodes with the same parent

```
<html>
  <head>
    <title>DOM Tutorial</title>
  </head>

  <body>
    <h1>What is DOM? </h1>
    <p>Hello world!</p>
  </body>
</html>
```



The DOM Programming Interface…

- The HTML DOM objects can be accessed with JavaScript
- The programming interface is the properties and methods of each object.
  - A property is a value that you can get or set (like changing the content of an HTML element).
  - A method is an action you can do (like add or deleting an HTML element)

JavaScript Basics

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

JavaScript Terminology

- JavaScript uses specialized terminology.
- Objects, Properties, Methods, Events, Functions, Values, Variables, Expressions, Operators.
  - **JavaScript** is a prototype-based **object-oriented** language

https://en.wikipedia.org/wiki/Prototype-based_programming

Prototype-based programming is a style of object-oriented programming in which behavior reuse (known as inheritance) is performed via a process of reusing existing objects that serve as prototypes.

JavaScript Objects

- Objects refers to windows, documents, images, tables, forms, buttons or links, etc.
- Objects should be named.
- Objects have properties that act as modifiers.

JavaScript Properties

- Properties are object attributes.
- Object properties are defined by using the object's name, a period, and the property name.

Ex: background color is expressed by:

```
document.bgcolor
```

object property

JavaScript Methods

- Methods are actions applied to particular objects.
- Methods are what objects can do

Ex:

```
document.write("Hello World");
```

object method

The “window” Object

- The window object represents a window containing a DOM document.
- Window is the object of browser; it is not the object of JavaScript.
  - created automatically by the browser.
- **Global functions** are methods of the **window** object.
- **Global variables** are properties of the **window** object.
- Since window is the global object, you can reference any properties of it with just the property name - so you do not have to write down **window**

Some Window Object Properties

Property	Description
closed	Returns a Boolean value indicating whether a window has been closed or not
console	Returns a reference to the Console object, which provides methods for logging information to the browser's console
document	Returns the Document object for the window
localStorage	Allows to save key/value pairs in a web browser. Stores the data with no expiration date
sessionStorage	Allows to save key/value pairs in a web browser. Stores the data for one session
navigator	Returns the Navigator object for the window

https://www.w3schools.com/jsref/obj_window.asp

Some Window Object Methods

alert()	Displays an alert box with a message and an OK button
close()	Closes the current window
confirm()	Displays a dialog box with a message and an OK and a Cancel button
open()	Opens a new browser window
setTimeout()	Calls a function or evaluates an expression after a specified number of milliseconds
clearTimeout()	Clears a timer set with setTimeout()

https://www.w3schools.com/jsref/obj_window.asp

The “document” Object

- When an HTML document is loaded into a web browser, it becomes a document object.
- The document object is the root node of the HTML document.

Document Object Properties and Methods

getElementById()	Returns the element that has the ID attribute with the specified value
getElementsByClassName()	Returns a HTMLCollection containing all elements with the specified class name
getElementsByName()	Returns a HTMLCollection containing all elements with a specified name
getElementsByTagName()	Returns a HTMLCollection containing all elements with the specified tag name
URL	Returns the full URL of the HTML document
write()	Writes HTML expressions or JavaScript code to a document
writeln()	Same as write(), but adds a newline character after each statement
forms()	Returns a collection of all <form> elements in the document

https://www.w3schools.com/jsref/dom_obj_document.asp

Document Object Properties and Methods

- document.**body** represents the <body> tag.
- innerHTML represents HTML contents of the node.

```
<!DOCTYPE html>
<html>
  <body>
    <h1 id="id">My First Page</h1>
    <script>
        document.getElementById("id").innerHTML = "Hello";
    </script>
  </body>
</html>
```

Method

Property

Hello

Examples

```
window.document.getElementById("header");
```

```
var element=document.getElementById("header");
element.innerHTML="New Header";

document.getElementById("p2").style.color="blue";
```

```
document.write('<p>Welcome!</p>');
```

```
document.body.innerHTML = "Hello"
```

```
document.body.style.background = 'red’
```

JavaScript Syntax

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

JavaScript Syntax…

- Unlike HTML, JavaScript is case sensitive.
- Dot Syntax is used to combine terms.

```
document.write("Hello World");
```

- Certain characters and terms are reserved.
- JavaScript is simple text (ASCII)

...JavaScript Syntax

- JavaScript ignores spaces, tabs, and newlines that appear in JavaScript programs.
- Semicolons are optional

```
<script>
    var1 = 10
    var2 = 20
    console.log(var1 + var2);
</script>
```

when formatted in a single line as follows, you must use semicolons

```
<script>
    var1 = 10; var2 = 20;
    console.log(var1 + var2);
</script>
```

Using Comments

- JavaScript supports both C-style and C++ style comments

```
<script>
     // single line comment

     /*
     This is block comment.
     This is block comment.
     */
 </script>
```

JavaScript Datatypes

- Primitive data types:
  - Number: 1, 2, 3, etc. (All numbers are represented as floating-point values)
  - String: characters enclosed in quotes. Ex: “web programming"
  - Boolean: true or false.
- null & undefined
  - null
    - null is an assignment value
  - undefined
    - Is as type
    - variable declared, but no value has been assigned
  - In JavaScript, undefined is a type, whereas null an object.
- composite data type
  - object

null & undefined Examples

```
<script>
    var var1;
    console.log(var1); //shows undefined
    console.log(typeof var1); //shows undefined

    var var2 = null;
    console.log(var2); //shows null
    console.log(typeof var2); //shows object
</script>
```

Practice with **js-sample2.html**

JavaScript Identifiers (Naming)

- First character must be a letter, or an underscore, or a $ sign
  - Numbers are not allowed as the first character.
- In the name, underscore and camel Case can be used:
  - first_name, last_name, master_card, inter_city
  - FirstName, LastName, MasterCard, InterCity (Upper Camel Case)
  - firstName, lastName, masterCard, interCity (Lower Camel Case)
- Hyphens are not allowed

Variables

- JavaScript is an **untyped** language.
  - means that a variable can hold a value of any data type
- The value type of a variable can change during the execution of a program
- Variables are created by declaration using the **var, let** (ES6) **or const**(ES6) commands with or without an initial value state.

Variable Examples

```
<script>
    var name = "Ali";
    console.log(name+" "+typeof name);
    var money;
    console.log(money+" "+typeof money);
    money = 2000.50;
    console.log(money+" "+typeof money);
</script>
```

```
Ali string
undefined undefined
2000.5 number
```

JavaScript Variable Scopes

- The scope of a variable is the region of your program in which it is defined.
- JavaScript variables have only two scopes:
  - **Global Variables** – A global variable has **global** scope which means it can be defined anywhere in your JavaScript code.
  - **Local Variables** – A local variable will be visible only within a **function** where it is defined. Function parameters are always local to that function.
- When a variable is declared without the var, let, or const keywords, they're automatically attached to the window object, making them available to every part of your web app.

ES6 Variables

ES6 introduces new ways to declare variables:

- let – creates a scope variable
  - Accessible only in its scope (restricted scope)

```
for(let number of [1, 2, 3, 4]){
 console.log(number);
}
//accessing number here throws exception
```

- const – creates a constant variable
  - Its value is read-only and cannot be changed
  - block-scoped, much like variables defined using the let statement

```
const MAX_VALUE = 16;
MAX_VALUE = 15; // throws exception
```

Variable Scope Examples

```
<script>
        var city = "istanbul";      // Declare a global variables
        var district="cekmekoy";
        console.log("City:"+city);
        function checkscope( ) {
            var city = "ankara";  // Declare a local variable
            console.log("City:"+city+" District:"+district);
        }
        checkscope();
        console.log("City:"+city);
</script>
```

```
City:istanbul
City:ankara District:cekmekoy
City:istanbul
```

Practice with **js-sample3.html**

Templated Strings in ES6

ES6 supports templated strings

- i.e. strings with placeholders:

```
let name = 'Murat';
let surname = 'Kara';
console.log('Name: ' + name + ' Surname: ' + surname);
```

Becomes

```
console.log(`Name: ${name} Surname: ${surname}`);
```

Output: Name: Murat Surname: Kara

Operators

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

JavaScript Operators

- Operators are used to handle variables.
- Types of operators with examples:
  - Arithmetic operators
  - Comparison operators
  - Logical operators
  - Control operators
  - Assignment and String operators.

Arithmetic Operators

- +, -, *, / (division), % (modulus)
  - Increment
    - ++a : increments and then uses the variable.
    - a++ : uses and then increments the variable.
  - Decrement
    - --a : decrements and then uses the variable.
    - a-- : uses and then decrements the variable.

Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Examples

```
var a = 33;
var b = 10;
var c = "Test";
```

```
result = a + b;
console.log(result); //43
result = a - b;
console.log(result);//23
result = a / b;
console.log(result);//3.3
result = a % b;
console.log(result);//3
result = a + b + c;
console.log(result);//43Test
```

```
console.log(a);//33
result = ++a;
console.log(result);//34
console.log(a);//34
console.log(b);//10
result = b++;
console.log(result);//10
console.log(b);//11
a+=b;
console.log(a);//45
```

Demo time: Practice with **js-sample4.html**

Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

```
var a = 33;
var b = "33";
console.log(a==b); //true
console.log(a===b);//false
```

Demo time: Practice with **js-sample4.html**

Logical Operators

Operator	Description
&&	logical and
||	logical or
!	logical not

Conditional Statements

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

Conditional branching: if

```
if (condition1) {
  // block of code to be executed if condition1 is true
} else if (condition2) {
  // block of code to be executed if the condition1 is false and
  // condition2 is true
} else {
  // block of code to be executed if the condition1 is false and
  // condition2 is false
}
```

Example

```
if (time < 10) {
     greeting = "Good morning";
} else if (time < 20) {
     greeting = "Good day";
} else {
     greeting = "Good evening";
}
```

Switch Statement

- A switch statement can replace multiple if checks.
- It gives a more descriptive way to compare a value with multiple variants.

```
switch (expression) {
  case value 1:
      statement(s)
      break;
  case value 2:
      statement(s)
      break;
  ...
  case value n:
      statement(s)
      break;
  default:
     statement(s)
}
```

Switch Statement Example

```
var grade = 'D';
switch (grade) {
   case 'A': document.write("Good job<br />");
       break;
   case 'B': document.write("Pretty good<br />");
       break;
   case 'C': document.write("Passed<br />");
       break;
   case 'D': document.write("Not so good<br />");
       break;
   case 'F': document.write("Failed<br />");
       break;
   default: document.write("Unknown grade<br />")
}
document.write("Exiting switch block");
```

If we do not add the break statement after the code statements, then the interpreter would execute each and every case after the matched case.

Practice with **js-sample5.html**

Loops

JavaScript

1. What is Java Script and Scripting Language
2. Adding JavaScript to a web page
3. HTML DOM
4. JavaScript Basics
5. JavaScript Syntax
6. Operators
7. Conditional Statements
8. Loops

Loop Types

- for
  - loops through a block of code several times
- for/in
  - loops through the **properties of an object**
- for/of loop
  - loops through the values of **an iterable objects**
- while
  - loops through a block of code while a specified condition is true
- do/while
  - also loops through a block of code while a specified condition is true
- Some others…

for Loop

```
for (initialization; test condition; iteration statement) {
    // code block to be executed
}
```

```
for (i = 0; i < 15; i+=2) {
     console.log("The number is " + i );
}
```

```
The number is 0
The number is 2
The number is 4
The number is 6
The number is 8
The number is 10
The number is 12
The number is 14
```

for…in Loop

```
var student = { fname: "Kerem", lname: "Karaca", id: 234 };
for (x in student) {
    console.log("prop name:"+x+" value:"+student[x])
}
```

```
prop name:fname value:Kerem
prop name:lname value:Karaca
prop name:id value:234
```

for…of Loop

```
var names = ["Kerem", "Kenan", "Kaya"];
var name;

for (name of names) {
    console.log(name);
}
```

```
Kerem
Kenan
Kaya
```

while loop

```
while (condition) {
  // code block to be executed
}
```

```
i = 5;
while (i > 0) {
    console.log("The number is " + i);
    i--;
}
```

```
The number is 5
The number is 4
The number is 3
The number is 2
The number is 1
```

do/while Loop

- This loop will execute the code block once, before checking if the condition is true

```
i = 5;
do{
    console.log("The number is " + i);
    i--;
} while (i >5);
```

```
The number is 5
```

Practice with **js-sample6.html**

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr